VecBench-EC2: Performance Benchmarking of Open-Source Vector Databases for RAG Systems on AWS EC2

CS5296 Cloud Computing, Spring 2026 — Technical Project

Group XX

QING Qiguo (59905956) LUO Yuxi (59570433)

Department of Computer Science, City University of Hong Kong

Abstract

Retrieval-Augmented Generation (RAG) systems depend on vector databases for efficient nearest-neighbor search over high-dimensional embeddings. Selecting an appropriate vector database involves trade-offs in ingestion speed, query latency, scalability, and cost. This paper presents VecBench-EC2, a systematic benchmark comparing three open-source vector databases—Milvus, Chroma, and Weaviate—on AWS EC2 using the SIFT1M dataset (128-dimensional vectors). We evaluate ingestion throughput, query latency percentiles (P50/P95/P99), concurrency scaling (1–16 threads), memory consumption, and total cost of ownership (TCO) at three dataset scales (10K, 100K, 1M vectors). All experiments run on a t2.large instance (2 vCPU, 8 GB RAM) under controlled conditions to ensure fair comparison. Our results reveal that Chroma achieves the lowest query latency (3.6 ms average) and highest concurrent throughput (5,479 QPS at 4 threads) at moderate scale, but fails at 1M vectors due to memory exhaustion. Milvus demonstrates superior scalability, handling 1M vectors with 35,370 vec/s ingestion throughput, though it exhibits higher per-query latency (290 ms) due to gRPC overhead. Weaviate offers balanced performance but encounters timeout failures at 1M scale. These findings provide practical guidance for deploying vector databases in resource-constrained cloud environments.

1 Introduction

The rapid adoption of Retrieval-Augmented Generation (RAG) architectures has elevated vector databases to a critical infrastructure component in modern AI systems [1]. A RAG pipeline encodes documents into high-dimensional embedding vectors, stores them in a vector database, and retrieves the most relevant vectors at query time to augment language model responses. The choice of vector database directly impacts system latency, throughput, and operational cost.

The vector database market has grown significantly, with both commercial (Pinecone, Zilliz Cloud) and open-source (Milvus, Chroma, Weaviate) solutions competing for adoption [2]. However, practitioners often lack controlled benchmarks to guide their selection, particularly in cloud environments where resource constraints and pricing affect deployment decisions.

This project addresses this gap by benchmarking three popular open-source vector databases on AWS EC2:

- Milvus v2.4 [3]: A distributed vector database with segment-based architecture, supporting multiple index types (IVF_FLAT, HNSW, FLAT) and strong consistency via etcd.
- Chroma (latest) [4]: A lightweight, developer-friendly embedding database designed for rapid prototyping, using an HTTP API.
- Weaviate v1.24 [5]: A cloud-native vector search engine with a GraphQL/REST API and built-in HNSW indexing.

Our benchmarking methodology follows established practices from ANN-Benchmarks [6] and VectorDBBench [7], with the following contributions:

1. A unified Python framework that benchmarks all three databases under identical conditions on the same EC2 instance.
2. Comprehensive evaluation covering ingestion, query latency percentiles, concurrency scaling, and system resource monitoring.
3. TCO analysis for deploying vector databases on AWS EC2.
4. Documentation of scalability limits on memory-constrained instances.

2 Background and Related Work

2.1 Vector Similarity Search

Vector databases perform approximate nearest neighbor (ANN) search over high-dimensional vectors using distance metrics such as L2 (Euclidean) or cosine similarity. The core challenge is maintaining sub-millisecond query latency as dataset size grows from thousands to millions of vectors. Index structures such as FLAT (brute-force), IVF (inverted file), and HNSW (hierarchical navigable small world) [8] offer different trade-offs between recall accuracy and search speed.

2.2 Related Benchmarks

ANN-Benchmarks [6] evaluates algorithmic performance of indexing libraries (FAISS, Annoy, etc.) but does not benchmark full database systems. VectorDBBench [7] by Zilliz provides a more comprehensive evaluation but targets high-end hardware. Our work fills the gap of benchmarking on resource-constrained cloud instances typical of cost-sensitive deployments, and additionally measures concurrency scaling and TCO.

2.3 Database Architectures

Table 1 summarizes the architectural differences among the three databases.

Table 1: Architectural comparison of benchmarked vector databases.

Feature	Milvus	Chroma	Weaviate
Docker containers	3 (etcd, minio, standalone)	1	1
API protocol	gRPC	HTTP/REST	REST/GraphQL
Default index	FLAT	HNSW	HNSW
Storage backend	minio (S3-compatible)	SQLite + Parquet	Built-in LSM
Consistency model	Strong/Bounded	Eventual	Eventual

3 Experimental Methodology

3.1 Fairness Rules

To ensure a fair comparison, all experiments adhere to the following rules:

1. Same EC2 instance for every database.
2. Same OS and Docker version (Ubuntu 22.04, Docker CE latest).
3. Same dataset (SIFT1M subsets from identical .npy files).
4. Fixed query set (100 queries randomly sampled from SIFT1M query vectors, seed=42).
5. Fixed random seed (seed=42) for all operations.
6. Same top_k=10 for all search benchmarks.
7. 1-second warm-up pause after initialization.
8. Each experiment repeated 3 times; results report the mean.
9. Sequential execution: one database at a time to avoid resource contention.
10. Fresh state: collection dropped and recreated between rounds.

3.2 Dataset

We use the SIFT1M dataset [9], a standard benchmark for ANN evaluation containing 1 million 128-dimensional vectors derived from SIFT feature descriptors. We prepare three subsets (10K, 100K, 1M vectors) and a fixed set of 1,000 query vectors, from which 100 are used per benchmark run.

3.3 Benchmark Framework

Our Python benchmark framework (benchmark.py) implements a unified BaseVectorDB interface with three adapters (MilvusDB, ChromaDB, WeaviateDB). The framework measures:

- Ingestion: Batch insert with database-specific batch sizes (Milvus: 10K, Chroma: 5K, Weaviate: object-level batching). Wall-clock time includes index building and flushing.
- Query latency: Sequential single-vector queries, recording per-query latency. Percentiles (P50, P95, P99) computed via numpy.percentile.
- Concurrency: Multi-threaded query execution using Python's ThreadPoolExecutor at 1, 4, 8, and 16 threads.
- System metrics: CPU and memory usage captured via psutil before/after insert and query phases.

3.4 Infrastructure

- EC2 instance: t2.large (2 vCPU, 8 GB RAM, burstable)
- Storage: 30 GB gp3 EBS volume
- OS: Ubuntu 22.04 LTS
- Docker: Docker CE with Compose v2
- Python: 3.10+ with pymilvus, chromadb, weaviate-client v3

4 Results and Analysis

4.1 Ingestion Performance

Figure 1 shows ingestion throughput across the three dataset scales. Key observations:

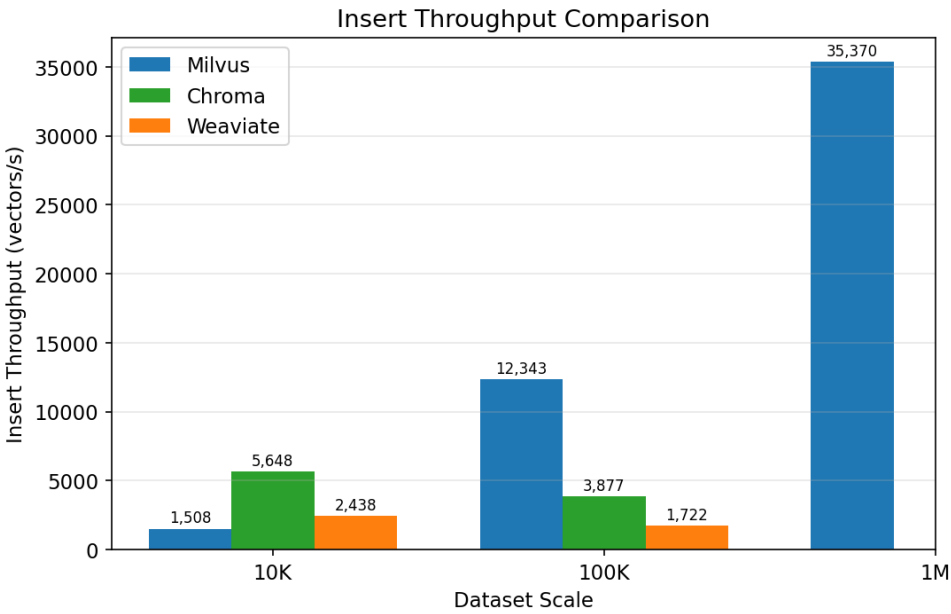

Figure 1: Insert throughput (vectors/second) by dataset scale. Higher is better.

Milvus exhibits a counter-intuitive scaling pattern: throughput increases from 1,508 vec/s (10K) to 12,343 vec/s (100K) to 35,370 vec/s (1M), because its per-vector amortized cost decreases as collection initialization and index creation overhead is spread over more vectors. Chroma leads at 10K (5,648 vec/s) but degrades to 3,877 vec/s at 100K and fails at 1M due to OOM (>5.3 GB). Weaviate shows consistently low throughput (2,438 → 1,722 vec/s) due to object-level batching and real-time HNSW index updates.

4.2 Query Latency

Figure 2 presents query latency distributions at each scale under sequential (1-thread) execution.

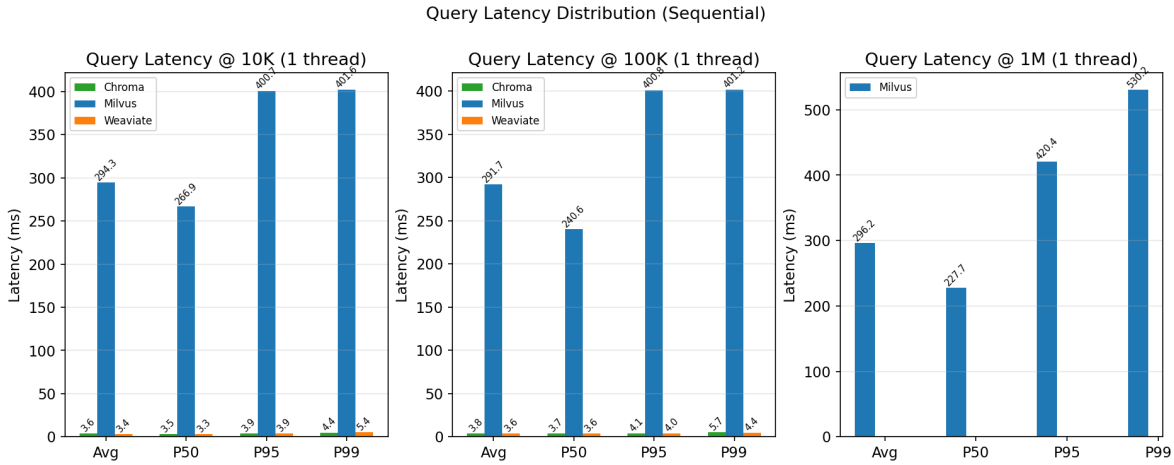

Figure 2: Query latency (Avg, P50, P95, P99) under sequential execution. Lower is better.

Chroma and Weaviate achieve sub-5ms average latency at both 10K and 100K due to their HNSW index ($O(\log n)$ complexity). Milvus exhibits 290–296 ms average latency—approximately 80× slower—due to (1) brute-force FLAT index L2 computation and (2) per-query gRPC serialization overhead. At 1M, P99 rises to 530 ms.

4.3 Concurrency Scaling

Figure 3 shows query throughput (QPS) as concurrency increases from 1 to 16 threads.

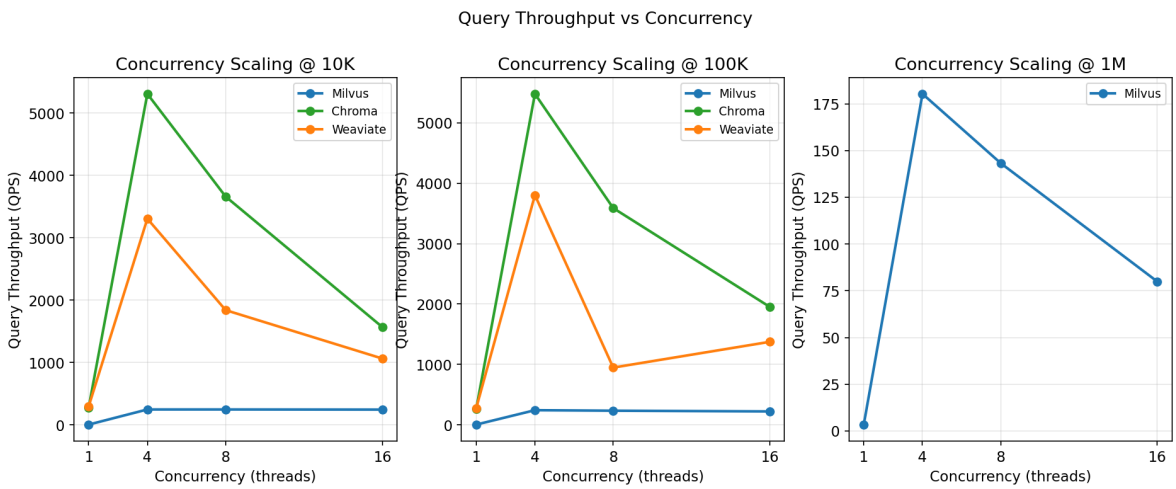

Figure 3: Query throughput (QPS) vs. concurrency level. Higher is better.

Chroma peaks at 4 threads (5,479 QPS at 100K), then degrades due to the Python GIL. Weaviate peaks similarly at 4 threads (3,803 QPS) but shows higher variance at 8–16 threads from goroutine scheduling contention. Milvus shows flat throughput (∼242 QPS at 100K) regardless of concurrency, bottlenecked by gRPC round-trips and brute-force FLAT index on 2 vCPUs.

4.4 Tail Latency Under Load

Figure 4 shows how P95 latency changes under increasing concurrency.

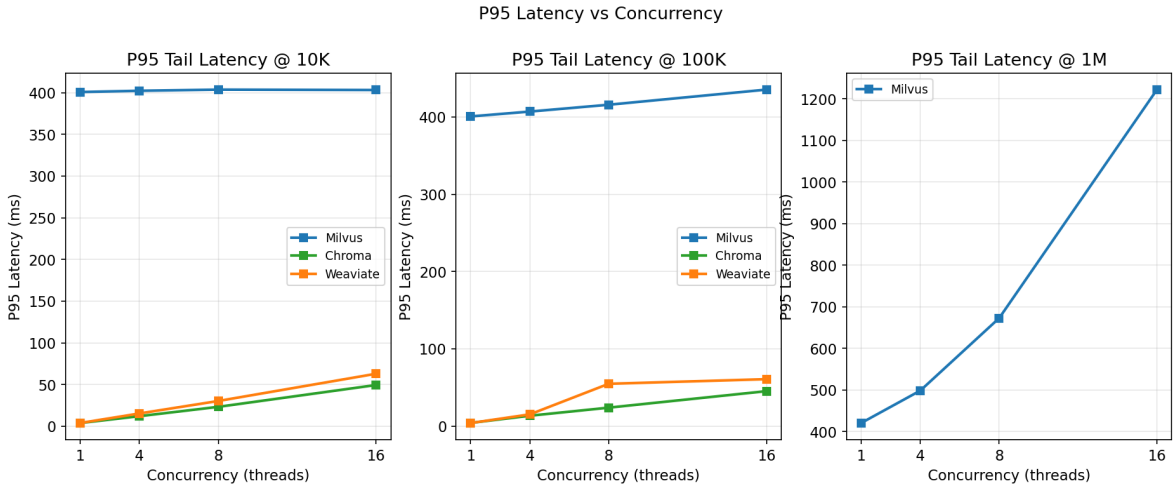

Figure 4: P95 tail latency vs. concurrency. Lower is better.

Chroma and Weaviate maintain P95 under 60 ms even at 16 threads for 10K/100K. Milvus P95 remains ∼400 ms at 100K but degrades to 1,222 ms at 1M/16 threads, indicating resource exhaustion.

4.5 Memory Consumption

Figure 5 shows system memory after ingestion. Milvus consumes 6,792 MB at 1M (3 containers), leaving only 1.2 GB for the OS. Chroma and Weaviate use 1,477–1,522 MB at 100K but exceed 8 GB during 1M ingestion, triggering the OOM killer. At 10K, all fit comfortably (741–1,007 MB).

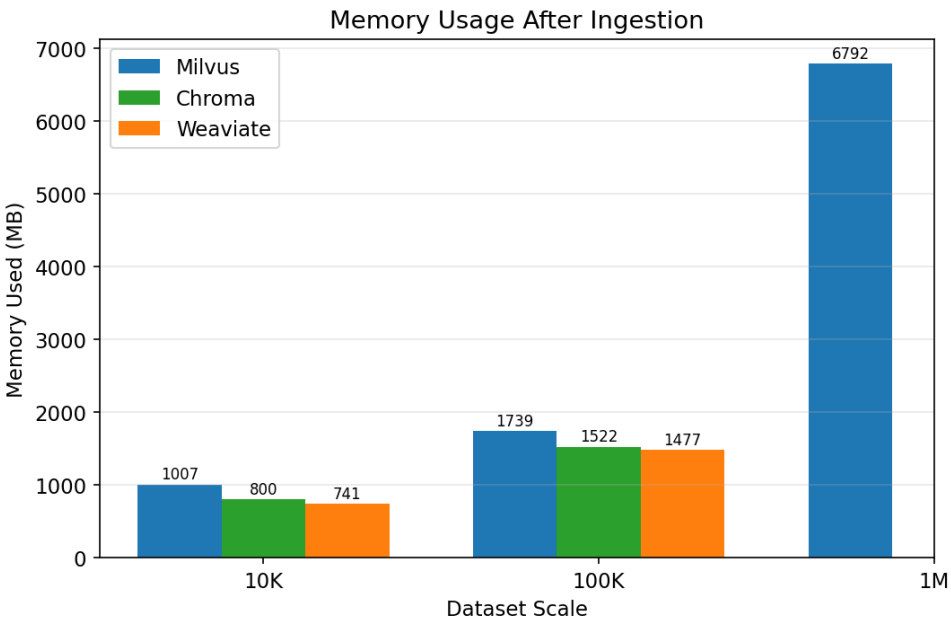

Figure 5: System memory usage after ingestion. Includes OS and Docker overhead.

4.6 Scalability Limits

Table 2 summarizes the success or failure of each database at each scale.

Table 2: Scalability results on t2.large (8 GB RAM).

Scale	Milvus	Chroma	Weaviate
10K	Pass	Pass	Pass
100K	Pass	Pass	Pass
1M	Pass	Fail (OOM)	Fail (OOM / Timeout)

Chroma's 1M failure is caused by its in-memory HNSW index exceeding available RAM. Weaviate fails with a ReadTimeout during batch ingestion, followed by the OOM killer terminating the container. Only Milvus, which offloads data to minio (disk-backed object storage), survives the 1M workload on 8 GB RAM.

5 Cost Analysis

Table 3 presents the monthly TCO for running a vector database on EC2 t2.large.

Table 3: Monthly Total Cost of Ownership (TCO) on AWS EC2 t2.large.

Component	Specification	Cost (USD/month)
EC2 compute	t2.large, on-demand	\$67.74
EBS storage	30 GB gp3	\$2.40
Data transfer	∼1 GB outbound	\$0.09
Total		\$70.23

The per-experiment cost is negligible: Milvus's full 3-scale benchmark (9 rounds) consumed approximately 9 minutes of EC2 time (\$0.014), Weaviate 4 minutes (\$0.006), and Chroma 2 minutes (\$0.003).

For production deployments requiring 1M+ vectors, practitioners would need at least a t3.xlarge (4 vCPU, 16 GB RAM, \$0.1664/hr = \$121.47/month) or r5.large (2 vCPU, 16 GB RAM, \$0.126/hr = \$91.98/month) to accommodate all three databases. For Milvus with its disk-backed architecture, the t2.large may suffice for up to ∼2M vectors with sufficient EBS.

6 Discussion

6.1 Deployment Guidelines

For prototyping with <100K vectors, Chroma offers sub-4ms latency and 5,479 QPS with single-container simplicity. For production at >100K, Milvus is the only option scaling to 1M on 8 GB RAM via disk-backed storage, at the cost of higher per-query latency. Weaviate provides a middle ground with low-latency REST/GraphQL APIs but limited insert performance at scale.

6.2 Threats to Validity

The t2.large burstable instance may deplete CPU credits during long experiments. Milvus uses FLAT index while Chroma/Weaviate default to HNSW—using HNSW on Milvus would reduce its query latency. Python's GIL limits true parallelism in concurrency tests, though queries execute in database server processes. Finally, SIFT1M is a single-distribution dataset; real RAG embeddings may exhibit different patterns.

7 Conclusion

This paper presented VecBench-EC2, a controlled benchmark of Milvus, Chroma, and Weaviate on AWS EC2 t2.large using the SIFT1M dataset at 10K/100K/1M scales. Our experiments demonstrate that Chroma excels at small-to-medium scale with sub-4ms query latency and over 5,000 QPS under moderate concurrency, while Milvus is the only database capable of handling 1M vectors on an 8 GB instance, achieving 35,370 vec/s ingestion throughput through its disk-backed storage architecture. These findings provide actionable guidance for practitioners selecting vector databases under cloud resource and budget constraints.

References

[1] P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in Proc. NeurIPS, 2020.

[2] J. Pan et al., "Survey of vector database management systems," VLDB Journal, vol. 33, pp. 1039–1068, 2024.

[3] J. Wang et al., "Milvus: A purpose-built vector data management system," in Proc. ACM SIGMOD, 2021, pp. 2614–2627.

[4] Chroma, "Chroma — the AI-native open-source embedding database," 2024. [Online]. Available: https://www.trychroma.com/

[5] SeMI Technologies, "Weaviate vector database," 2024. [Online]. Available: https://weaviate.io/

[6] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms," Information Systems, vol. 87, 2020.

[7] Zilliz, "VectorDBBench: Open-source vector database benchmark tool," 2024. [Online]. Available: https://github.com/zilliztech/VectorDBBench

[8] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," IEEE Trans. PAMI, vol. 42, no. 4, pp. 824–836, 2020.

[9] H. Jégou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," IEEE Trans. PAMI, vol. 33, no. 1, pp. 117–128, 2011.

[10] Amazon Web Services, "Amazon EC2 on-demand pricing," 2026. [Online]. Available: https://aws.amazon.com/ec2/pricing/on-demand/

Artifact Appendix

A.1 Abstract

This artifact contains the complete benchmark framework, Docker Compose configurations, dataset preparation scripts, and analysis tools used in this paper. All experiments can be reproduced on an AWS EC2 t2.large instance running Ubuntu 22.04.

A.2 Artifact Location

- GitHub Repository: https://github.com/YOUR_USERNAME/vecbench-ec2
- License: MIT

A.3 Requirements

Hardware:

- AWS EC2 t2.large or larger (2+ vCPU, 8+ GB RAM)
- 30 GB EBS gp3 storage

Software:

- Ubuntu 22.04 LTS
- Docker CE 24+ with Compose v2
- Python 3.10+
- Dependencies: see requirements.txt

A.4 Setup and Installation

```
# 1. Clone the repository
git clone https://github.com/YOUR_USERNAME/vecbench-ec2.git
cd vecbench-ec2

# 2. Run environment setup (installs Docker, Python, venv)
chmod +x scripts/setup_env.sh
./scripts/setup_env.sh
# Log out and back in for Docker group, then:
source venv/bin/activate

# 3. Download and prepare dataset
python downloadDataset.py
python scripts/prepare_sift_subsets.py
```

A.5 Running the Benchmark

```
# Run all experiments (takes ~1 hour on t2.large)
nohup bash scripts/run_phase3_all.sh > results/phase3.log 2>&1 &

# Or run individual databases:
```

```
cd docker/milvus && docker compose up -d && cd ../..
python benchmark.py --db milvus --dataset data/processed/sift_10k.npy \
    --scale 10K --threads 1,4,8,16 --repeat 3 --num-queries 100
cd docker/milvus && docker compose down && cd ../..
```

A.6 Generating Charts and Analysis

```
python scripts/analyze_results.py
# Output: bench/charts/*.png and aggregated_results.csv
```

A.7 Expected Results

The benchmark produces CSV files in results/ with columns for insert time, query latency percentiles, concurrency level, and system metrics. At 100K scale, expected approximate values:

- Milvus: insert ∼12,300 vec/s, query avg ∼292 ms
- Chroma: insert ∼3,877 vec/s, query avg ∼3.8 ms
- Weaviate: insert ∼1,722 vec/s, query avg ∼3.6 ms

A.8 File Structure

```
vecbench-ec2/
  benchmark.py              # Main benchmark framework
  downloadDataset.py        # Download SIFT1M from HuggingFace
  requirements.txt          # Python dependencies
  docker/
    milvus/docker-compose.yml
    chroma/docker-compose.yml
    weaviate/docker-compose.yml
  scripts/
    setup_env.sh            # EC2 environment setup
    prepare_sift_subsets.py # Generate .npy subsets
    smoke_test.py           # Connectivity verification
    collect_system_metrics.py
    run_phase3_all.sh       # Full experiment automation
    analyze_results.py      # Data analysis & charts
  bench/
    results/                # Raw CSV results
    charts/                 # Generated visualizations
  data/
    raw/                    # Original SIFT1M fvecs
    processed/              # 10K/100K/1M .npy subsets
```